Document number: P0433R2
Date: 2017-03-03
Reply-To:
Mike Spertus, Symantec (mike_spertus@symantec.com)
Walter E. Brown (webrown.cpp@gmail.com)
Stephan T. Lavavej (stl@exchange.microsoft.com)
Audience: {Library Evolution, Library} Working Group

Toward a resolution of US7 and US14: Integrating template deduction for class templates into the standard library

Introduction

National body comments US7 and US14 request analysis of the standard library to determine what changes might be desirable in light of the C++17 adoption of P0091R3 (Template argument deduction for class templates (rev. 6)). In this paper, we perform such an analysis and recommend wording changes for the standard library clauses.

Background

General background is provided in P0433R0. We list here new items not reflected in that paper.

- A sample implementation is available at https://github.com/mspertus/p0433. The primary impressions from implementation experience are that classes focused on a particular "application domain" generally work just as one would hope without any changes, which we believe bodes well for C++ programmers developing applications. E.g.,

  ```
  string needle = "foo";
  auto bms = boyer_moore_searcher(needle.begin(), needle.end());
  gamma_distribution gdf{2.5f, .7f};
  ```

  while heavily-metaprogrammed STL classes (e.g., `vector`) generally require some explicit deduction guides but that generally proved straightforward. Adapting the entire standard library took about 40 hours, most of which was spent on writing test cases.
- P0091R4 discusses some possible language extensions. Our recommendations in this paper for integrating template deduction for class templates are not affected in any way by the adoption or non-adoption of the extensions in that paper. However, it should be noted that these language extensions were motivated by experience with the standard library. See the next bullet as well as the discussion of `valarray` below for more information.
- As has been noted multiple times on the reflector, scenarios like the following can arise with standard library types that act as containers for values (e.g., `optional`):

  ```
  optional o(3);  // Deduces optional<int>
  optional o2 = o;  // Deduces optional<int>
  ```

  However, note that writers of template libraries may have to continue to specify some object constructions in the traditional fashion as illustrated here:

  ```
  template<typename T> void foo(T t)
  {
    optional o = t; // T=optional<int> => o is optional<int>, which may not be desired in generic code
    optional<T> o2 = t; // o2 is optional<optional<int>>
  }
  ```

  We feel this is the right tradeoff as application programmers consuming template libraries get full benefit of deduction and template library writers are not worse off than in C++14. P0091R4 suggests a language extension to extend deduction to the example above. However, note that this does not impact the present proposal.

Wording

The remainder of this paper walks through the standard library clauses and proposes wording changes to take advantage of template deduction for class templates. Even where no wording changes are required, we sometimes call out classes that benefit from implicitly generated deduction guides. Note that library vendors may choose to add additional explicit deduction guides to maintain the "as-if" rule. See §21 below for an example of this.

§17 [library]

Modify 17.5.4.2.1p2 [namespace.std] as follows:

- an explicit or partial specialization of any member class template of a standard library class or class template.
- a deduction guide for any standard library class template.

This clause requires no deduction guides because it specifies only C libraries and so no class templates are involved.

§18 [language.support]

No changes are required to §18 [language.support].

Rationale: All of the headers in §18 are listed in “Table 30 — Language support library summary” in §18.1 [support.general]:

	Subclause	Header(s)
18.2	Common definitions	`<cstddef>`

18.3	Implementation properties	`<limits>`
		`<climits>`
		`<cfloat>`

18.4	Integer types	`<cstdint>`
18.5	Start and termination	`<cstdlib>`
18.6	Dynamic memory management	`<new>`
18.7	Type identification	`<typeinfo>`
18.8	Exception handling	`<exception>`
18.9	Initializer lists	`<initializer_list>`

18.10	Other runtime support	`<csignal>`
		`<csetjmps>`
		`<cstdalign>`
		`<cstdarg>`
		`<cstdbool>`
		`<cstdlib>`

`<limits>`

The only class template in `<limits>` is `numeric_limits`. It has no non-default constructors (and only static members), so no changes are needed.

`<intializer_list>`

The <initializer_list> class template has only a default constructor and “magic” construction (from braced initializer lists) that already does template constructor deduction, so no changes are required.

Remaining headers in this clause

None of the `<c…>` headers or `<new>`, `<typeinfo>`, or `<exception>` define class templates, so they are unaffected by template constructor deduction.

§19 [diagnostics]

No changes are required to §19 [diagnostics].

Rationale: All of the headers in §19 are listed in “Table 31 — Diagnostics library summary” in §19.1:

	Subclause	Header(s)
19.2	Exception classes	`<stdexcept>`
19.3	Assertions	`<cassert>`
19.4	Error numbers	`<cerrno>`
19.5	System error support	`<system_error>`

`<stdexcept>`, `<cassert>`, `<cerrno>`

None of these define any class templates, so they are unaffected by template constructor deduction.

<system_error>

This header defines the class templates `is_error_code_enum`, `is_error_condition_enum`, and specializations of `std::hash`, all of which have only default constructors.

§20 [utilities]

All of the headers in §20 are listed in “Table 32 — General utilities library summary” in §20.1:

	Subclause	Header(s)
20.2	Utilities components	`<utility>`
20.3	Compile-time integer sequences	`<utility>`
20.4	Pairs	`<utility>`
20.5	Tuples	`<tuple>`
20.6	Optional objects	`<optional>`
20.7	Variants	`<variant>`
20.8	Storage for any type	`<any>`
20.9	Fixed-size sequences of bits	`<bitset>`
20.10	Memory	`<memory>` `<cstdlib>`
20.11	Smart pointers	`<memory>`
20.12	Memory resources	`<memory_resource>`
20.13	Scoped allocators	`<scoped_allocator>`
20.14	Function objects	`<functional>`
20.15	Type traits	`<type_traits>`
20.16	Compile-time rational arithmetic	`<ratio>`
20.17	Time utilities	`<chrono>` `<ctime>`
20.18	Type indexes	`<typeindex>`
20.19	Execution policies	`<execution>`

§20.2 [utility]

No changes are required in this subclause as it specifies no class templates.

§20.3 [intseq]

No changes are required in §20.3. Although `integer_sequence` is a class template, it has no non-default constructors. Interestingly, while the name `make_integer_sequence` “looks like” a make function, which is what the paper is trying to replace, it is actually a type and therefore a different beast altogether.

§20.4 [pairs]

Per LWG, we no longer propose that the pair constructor unwrap `reference_wrapper`. We do add the following guide which handles some cases missed by the implicit guide, including: Non-copyable arguments (`tuple<const T&...>` is excluded from overload resolution for non-copyable types) and array to pointer conversion. This draft adopts Zhihao Yuan's suggestion of taking template parameters by value to automatically get the decay.

Make the following change to the definition of `std::pair` in §20.4.2 [pairs.pair]:

```
        void swap(pair& p) noexcept(see below);
        };

  template<class T1, class T2>
  pair(T1, T2) -> pair<T1, T2>;
}
```

§20.5 [tuple]

`tuple` is handled similarly to `pair`. To accomplish that, make the following change to the definition of `std::tuple` in §20.5.2

[tuple.tuple]:

```
    void swap(tuple&) noexcept(see below);
    };
  template<class... UTypes>
    tuple(UTypes...) -> tuple<UTypes...>;
  template<class T1, class T2>
    tuple(pair<T1, T2>) -> tuple<T1, T2>;
  template<class Alloc, class... UTypes>
    tuple(allocator_arg_t, Alloc, UTypes...) -> tuple<UTypes...>;
  template<class Alloc, class T1, class T2>
    tuple(allocator_arg_t, Alloc, pair<T1, T2>) -> tuple<T1, T2>;
  template<class Alloc, class... UTypes>
    tuple(allocator_arg_t, Alloc, tuple<UTypes...>) -> tuple<UTypes...>;
}
```

§20.6 [optional]

`optional` is handled similarly to pair. Modify §20.6.3 [optional.optional] as follows:

```
  T *val; // exposition only
};

template<class T> optional(T) -> optional<T>;
```

Note: See the Background above for additional discussion of `optional`.

§20.7 [variant]

Because of P0510R0, `variant` is no longer SFINAE friendly and does not work with constructor type deduction. Note that we do not lose too much from this because `variant`'s *raison d'etre* is their ability to hold types beyond those they were initialized with.

§20.8 [any]

No changes are required in §20.8 as it defines no class templates.

§20.9 [template.bitset]

No changes required in this subclause. It would be tempting to have `bitset("01101"s)` deduce `bitset<5>`. However, we feel this is best left for a future proposal, likely after a clearer consensus around compile-time strings has been achieved.

§20.10 [memory]

`allocator` requires no changes as only the copy constructor is appropriate for deduction.

§20.11 [smartptr]

The primary question we considered was whether code like the following should be valid:

```
int *ip = new int();
unique_ptr uip{ip};  // Deduce unique_ptr<int>
```

On the positive side, the code is natural, useful (although not at object creation, when `make_unique` is preferable), and we have already received a request from the community for it. The downside is that it will fail to deduce an array type

```
int *ip = new int[5];
unique_ptr uip{ip};  // Still deduces unique_ptr<int>
```

We choose not to allow this problem, so we require that such deductions be ill-formed.

Modify §20.11.1.2.1p8 [unique.ptr.single.ctor] as follows:

> *Remarks:* If this constructor is instantiated with a pointer type or reference type for the template argument `D`, the program is ill-formed. If class template argument deduction (13.3.1.8) would select the function template corresponding to this constructor, then the program is ill-formed.

Add the following paragraph after §20.11.1.2.1p14 [unique.ptr.single.ctor]:

> *Remarks:*If class template argument deduction (13.3.1.8) would select a function template corresponding to either of these constructors, then the program is ill-formed.

We handle `shared_ptr` the same as `unique_ptr`. Because the an implicit guides from `T *` are non-deducible, we do not need to add

special wording for those constructors. We do not include a guide for the aliasing constructor as unnecessary.

Make the following modifications to §20.11.2.2 [util.smartptr.shared]:

```
    template<class U> bool owner_before(const shared_ptr<U>& b) const;
    template<class U> bool owner_before(const weak_ptr<U>& b) const;
  };

  template <class T> shared_ptr(weak_ptr<T>) ->  shared_ptr<T>;
  template <class T, class D> shared_ptr(unique_ptr<T, D>) ->  shared_ptr<T>;
```

For `weak_ptr`, make the following modification to §20.11.2.3 [util.smartptr.weak]

```
    template<class U> bool owner_before(const weak_ptr<U>& b) const;
  };

  template<class T> weak_ptr(shared_ptr<T>) -> weak_ptr<T>;
```

For completeness, `owner_less` relies on implicit deduction guides. `enable_shared_from_this` does not really require any guides, but the implicit ones certainly do no harm.

§20.12 [memory.resource]

No changes are required in this subclause as the `polymorphic_allocator<Tp>` constructors do not contain any information about `Tp` (except when constructed from other `polymorphic_allocator` objects, in which case deduction is properly done without any changes required).

§20.13 [allocator.adaptor]

We supply an explicit deduction guide so that `scoped_allocator_adaptor` can deduce its outer allocator. At the end of the definition of class `scoped_allocator_adaptor` in §20.13.1 [allocator.adaptor.syn], add

```
    scoped_allocator_adaptor select_on_container_copy_construction() const;
  };

template<class OuterAlloc, class... InnerAllocs> scoped_allocator_adaptor(OuterAlloc, InnerAllocs...)
  -> scoped_allocator_adaptor<OuterAllocator, InnerAllocs...>;
```

If a different outer allocator is desired, then it can still be specified explicitly.

§20.14 [function.objects]

`reference_wrapper` requires the following change to make sure `reference_wrapper(T&)` doesn't beat the copy constructor due to better conversions. Add the following deduction guide at the end of the definition of class `reference_wrapper` in §20.14.5 [refwrap]

```
    operator() (ArgTypes&&...) const;
  };

  template<typename T>
  reference_wrapper(reference_wrapper<T>) -> reference_wrapper<T>;
```

We suggest allowing `function` to deduce its template argument when initialized by a function. While it is tempting to add deduction guides for member pointers, we no longer do so as there is a question about whether the first objects should be a pointer or a reference. We are interested in committee feedback. Note that we can always add this post-c++17.Add the following deduction guide at the end of the class definition for `function` in §20.14.12.2 [func.wrap.func]

```
    template<class T> const T* target() const noexcept;

  };

  template<class R, class... ArgTypes>
    function(R(*)(ArgTypes...)) -> function<R(ArgTypes...)>;

  template<class F>
  function(F) -> function<see below>;
```

Add a paragraph after §20.14.13.2.1p11 [func.wrap.func.con]

```
  template<class F>
  function(F) -> function<see below>;
```

Remarks:

This deduction guide participates in overload resolution only if `&F::operator()` is well-formed when treated as an unevaluated operand. In that case, if `decltype(&F::operator())` is of the form `R(G::*)(A...)` *cv* `&`$_{opt}$ `noexcept`$_{opt}$ for a class type `G`, then the deduced type shall be `function<R(A...)>`.

[*Example:*

```
void f() {
  int i{5};
  function g = [&](double) { return i; }; // Deduces function<int(double)>
}
```

—end example]

Add a new paragraph after 20.14.13.2p3 [func.wrap.func]:

[*Note:* The types deduced by the deduction guides for `function` may change in future versions of this International Standard. *—end note*]

Note: There are some arbitrary choices in the deductions for member pointers. We would be interested to see if the comittee agrees with having deduction guides for member pointers in addition to function pointers, and if so, whether these are the right choices.

Since template constructor deduction can be used to construct objects of type `default_searcher`, `boyer_moore_searcher`, and `boyer_moore_horspool_searcher`, we propose getting rid of the searcher creation functions by modifying §20.14p2 as follows:

```
  // 20.14.13 searchers:
  template<class ForwardIterator, class BinaryPredicate = equal_to<>>
    class default_searcher;
  template<class RandomAccessIterator,
             class Hash = hash<typename iterator_traits<RandomAccessIterator>::value_type>,
                          class BinaryPredicate = equal_to<>>
    class boyer_moore_searcher;
  template<class RandomAccessIterator,
             class Hash = hash<typename iterator_traits<RandomAccessIterator>::value_type>,
                          class BinaryPredicate = equal_to<>>
    class boyer_moore_horspool_searcher;

- template<class ForwardIterator, class BinaryPredicate = equal_to<>>
- default_searcher<ForwardIterator, BinaryPredicate>
- make_default_searcher(ForwardIterator pat_first, ForwardIterator pat_last,
-                       BinaryPredicate pred = BinaryPredicate());
- template<class RandomAccessIterator,
-            class Hash = hash<typename iterator_traits<RandomAccessIterator>::value_type>,
-                         class BinaryPredicate = equal_to<>>
- boyer_moore_searcher<RandomAccessIterator, Hash, BinaryPredicate>
- make_boyer_moore_searcher(
-   RandomAccessIterator pat_first, RandomAccessIterator pat_last,
-         Hash hf = Hash(), BinaryPredicate pred = BinaryPredicate());
- template<class RandomAccessIterator,
-            class Hash = hash<typename iterator_traits<RandomAccessIterator>::value_type>,
-                         class BinaryPredicate = equal_to<>>
- boyer_moore_searcher<RandomAccessIterator, Hash, BinaryPredicate>
- make_boyer_moore_horspool_searcher(
-     RandomAccessIterator pat_first, RandomAccessIterator pat_last,
-         Hash hf = Hash(), BinaryPredicate pred = BinaryPredicate());

  // 20.14.14, hash function primary template:
```

Delete §20.14.13.1.1 [func.searchers.default.creation]

~~2014.13.2.1 `boyer_moore_searcher` creation functions [func.searcher.boyer_moore.creation]~~

```
- template<class ForwardIterator, class BinaryPredicate = equal_to<>>
- default_searcher<ForwardIterator, BinaryPredicate>
- make_default_searcher(ForwardIterator pat_first, RandomAccessIterator pat_last,
-                       BinaryPredicate pred = BinaryPredicate());
```

~~*Effects:* Equivalent to:~~

```
- return default_searcher<ForwardIterator, BinaryPredicate>(pat_first, pat_last, pred);
```

Delete §20.14.13.2.1 [func.searchers.boyer_moore.creation]

~~**2014.13.2.1 `boyer_moore_searcher` creation functions [func.searcher.boyer_moore.creation]**~~

```
template<class RandomAccessIterator,
          class Hash = hash<typename iterator_traits<RandomAccessIterator>::value_type>,
          class BinaryPredicate = equal_to<>>
boyer_moore_searcher<RandomAccessIterator, Hash, BinaryPredicate>
make_boyer_moore_searcher(
   RandomAccessIterator pat_first, RandomAccessIterator pat_last,
          Hash hf = Hash(), BinaryPredicate pred = BinaryPredicate());
```

~~*Effects:* Equivalent to:~~

```
return boyer_moore_searcher<RandomAccessIterator, Hash, BinaryPredicate>(
          pat_first, pat_last, hf, pred);
```

Delete §20.14.13.3.1 [func.searchers.boyer_moore_horspool.creation]

~~**2014.13.2.1 `boyer_moore_searcher` creation functions [func.searcher.boyer_moore.creation]**~~

```
template<class RandomAccessIterator,
          class Hash = hash<typename iterator_traits<RandomAccessIterator>::value_type>,
          class BinaryPredicate = equal_to<>>
boyer_moore_horspool_searcher<RandomAccessIterator, Hash, BinaryPredicate>
make_boyer_moore_horspool_searcher(
   RandomAccessIterator pat_first, RandomAccessIterator pat_last,
          Hash hf = Hash(), BinaryPredicate pred = BinaryPredicate());
```

~~*Effects:* Equivalent to:~~

```
return boyer_moore_searcher<RandomAccessIterator, Hash, BinaryPredicate>(
          pat_first, pat_last, hf, pred);
```

§20.15 [meta]

Since the classes in this subclause are designed primarily for compile-time metaprogramming, they do not have constructors that require special treatment as a result of template deduction for constructors.

§20.16 [ratio]

Since the classes in this subclause are designed primarily for compile-time metaprogramming, they do not have constructors that require special treatment as a result of template deduction for constructors.

§20.17 [time]

No wording changes necessary for §20.17 to leverage template deduction for constructors. Note that LEWG voted against adding a deduction guide to support expressions like `duration(5L)`.

§20.18 [type.index]

There are no class templates defined in this subclause, so nothing to do.

§20.19 [execpol]

The only class template in this subclause is the type trait `is_execution_policy`, so no changes are required as described in §20.15 above.

§21 [strings]

The string library can take advantage of template argument deduction for constructors with few wording changes. For example, `basic_string_view("foo")` correctly deduces that `charT` is `char`.

It is worth noting that a library vendor may choose to make additional deduction guides in accordance with the “as-if” rule (§1.9 footnote 5) as illustrated by the following example. Suppose a vendor has previously leveraged the “as-if” rule to implement the constructor

```
basic_string<charT, traits, Allocator>::basic_string(const charT* str, const Allocator& a = Allocator());
```

using the equivalent but non-deducible type `value_type` in place of `charT` as follows:

```
basic_string<charT, traits, Allocator>::basic_string(const value_type* str, const Allocator& a = Allocator());
```

The vendor could maintain this convention while continuing to satisfy the “as-if” rule in C++17 by adding the following *deduction-guide* to their implementation:

```
template<class charT, class traits = char_traits<charT>, class Allocator = allocator<charT>>
  basic_string(const charT*, Allocator = Allocator()) -> basic_string<charT, Allocator>;
```

Add the following after the end of the definition of class `basic_string` in §21.3.2 [basic.string]

```
    int compare(size_type pos1, size_type n1,
                const charT* s, size_type n2) const;
  };

  template<class InputIterator, class Allocator = allocator<typename iterator_traits<InputIterator>::value_type>>
  basic_string(InputIterator, InputIterator, Allocator = Allocator())
    -> basic_string<typename iterator_traits<InputIterator>::value_type,
                    char_traits<typename iterator_traits<InputIterator>::value_type>, Allocator>;
```

In §21.3.2.2 [string.cons], insert the following between paragraphs 19 and 20:

```
template<class InputIterator, class Allocator = allocator<typename iterator_traits<InputIterator>::value_type>>
basic_string(InputIterator, InputIterator, Allocator = Allocator())
    -> basic_string<typename iterator_traits<InputIterator>::value_type,
                    char_traits<typename iterator_traits<InputIterator>::value_type>, Allocator>;
```

> *Remarks:* Shall not participate in overload resolution if `InputIterator` is a type that does not qualify as an input iterator, or if `Allocator` is a type that does not qualify as an allocator [sequence.reqmts].

§22 [localization]

Since `wstring_convert` and `wbuffer_convert` are deprecated, we choose not to create deduction guides for them.

§23 [containers]

All of the headers in §23 are listed in “Table 102 — Containers library summary” in §23.1:

	Subclause	Header(s)
23.2	Requirements	

23.3	Sequence containers	`<array>`
		`<deque>`
		`<forward_list>`
		`<list>`
		`<vector>`

23.4	Associative containers	`<map>` `<set>`
23.5	Unordered associative containers	`<unordered_map>` `<unordered_set>`
23.6	Container adaptors	`<queue>` `<stack>`

Wording changes are needed in §23 for two purposes

1. Add deduction guides for constructing containers from iterators as described in “Explicitly specified Deduction Guides” in P0091R3
2. Prevent allocators from confusing things (as is their wont). As Zhihao Yuan has pointed out, with only implicit guides, `vector(5, allocator<int>())` would, presumably unintentionally, deduce `vector<allocator<int>>`

but we have to make those changes many times as follows:

§23.2 [container.requirements]

Modify the end of §23.2.3/13 [sequence.reqmts] and the following paragraph as follows:

> are called with a type `InputIterator` that does not qualify as an input iterator, then those functions shall not participare in overload resolution.
>
> — A deduction guide for a sequence container shall not participate in overload resolution if it has an `InputIterator` template parameter that is called with a type that does not qualify as an input iterator, or if it has an `Allocator` template parameter that is called with a type that does not qualify as an allocator.
>
> The extent to which an implementation determines that a type cannot be an input iterator is unspecified, except that

as a minimum integral types shall not qualify as input iterators. Likewise, the extent to which an implementation determines that a type cannot be an allocator is unspecified, except that as a minimum a type `A` not satisfying both of the following conditions shall not qualify as an allocator:

- The *qualified-id* `A::value_type` is valid and denotes a type [temp.deduct]
- The expression `declval<A&>().allocate(size_t{})` is well-formed when treated as an unevaluated operand

Add a paragraph to the end of §23.2.6 [associative.reqmts]

A deduction guide for an associative container shall not participate in overload resolution if any of the following are true:

- It has an `InputIterator` template parameter that is called with a type that does not qualify as an input iterator
- It has an `Allocator` template parameter that is called with a type that does not qualify as an allocator
- It has a `Compare` template parameter that is called with a type that qualifies as an allocator

Add a paragraph to the end of §23.2.7 [unord.req]

A deduction guide for an unordered associative container shall not participate in overload resolution if any of the following are true:

- It has an `InputIterator` template parameter that is called with a type that does not qualify as an input iterator
- It has an `Allocator` template parameter that is called with a type that does not qualify as an allocator
- It has a `Hash` template parameter that is called with an integral type or a type that qualifies as an allocator
- It has a `Pred` template parameter that is called with a type that qualifies as an allocator

Note: The reason to ensure that `Hash` is not integral is to keep it from matching the `size_type` that some constructors use to specify the number of hash buckets.

§23.3 [sequences]

For `std::array`, based on considerations from p0511r1, at the end of the definition of class `array` in 23.3.7.1 [array.overview], we add the following *deduction-guide*:

```
    constexpr const T * data() const noexcept;
  };

  template <class T, class... U>
    array(T, U...) -> array<T, 1 + sizeof...(U)>;
}
```

Add the following paragraph to the end of 23.3.7.2 [array.cons]

```
template <class T, class... U>
  array(T, U...) -> array<T, 1 + sizeof...(U)>;
```

Requires: `(is_same_v<T, U> && ...)` is `true`. Otherwise the program is ill-formed.

At the end of the definition of class `deque` in §23.3.8.1 [deque.overview], add the following *deduction-guide*:

```
    void clear() noexcept;
  };

  template <class InputIterator, class Allocator = allocator<typename iterator_traits<InputIterator>::value_type>>
  deque(InputIterator, InputIterator, Allocator = Allocator())
    -> deque<typename iterator_traits<InputIterator>::value_type, Allocator>;
```

At the end of the definition of class `forward_list` in §23.3.9.1 [forwardlist.overview], add the following *deduction-guide*:

```
    void reverse() noexcept;
  };

  template <class InputIterator, class Allocator = allocator<typename iterator_traits<InputIterator>::value_type>>
  forward_list(InputIterator, InputIterator, Allocator = Allocator())
    -> forward_list<typename iterator_traits<InputIterator>::value_type, Allocator>;
```

At the end of the definition of class `list` in §23.3.10.1 [list.overview], add the following *deduction-guide*s:

```
    void reverse() noexcept;
  };

  template <class InputIterator, class Allocator = allocator<typename iterator_traits<InputIterator>::value_type>>
```

```
list(InputIterator, InputIterator, Allocator = Allocator())
  -> list<typename iterator_traits<InputIterator>::value_type, Allocator>;
```

At the end of the definition of class `vector` in §23.3.11.1 [vector.overview], add the following *deduction-guide*:

```
  void clear() noexcept;
};

template <class InputIterator, class Allocator = allocator<typename iterator_traits<InputIterator>::value_type>>
vector(InputIterator, InputIterator, Allocator = Allocator())
  -> vector<typename iterator_traits<InputIterator>::value_type, Allocator>;
```

Note that with the above guides `vector<bool>` behaves properly as well.

§23.4 [associative]

First, we note that associative containers cannot always deduce their template parameters from an initializer list as illustrated by the following code.

```
map m = {{"foo", 2}, {"bar", 3}, {"baz", 4}}; // Error: initializer_list not reified in type system
map m2 = initializer_list<pair<char const *, int>>({{"foo", 2}, {"bar", 3}, {"baz", 4}}); // OK
```

Hopefully, this may be addressed by a future language extension in the post-C++17 timeline. Note also that we apply `remove_const_t` to the keys in order to find the proper comparator (or hash in case of unordered containers).

Add a paragraph to §23.4.1 [associative.general]

> The following exposition only type aliases may appear in deduction guides for associative containers:

```
template<class InputIterator>
using iter_key_t = remove_const_t<typename iterator_traits<InputIterator>::value_type::first_type>;  // exposition only
template<class InputIterator>
using iter_val_t = typename iterator_traits<InputIterator>::value_type::second_type;  // exposition only
template<class InputIterator>
using iter_to_alloc_t = pair<add_const_t<typename iterator_traits<InputIterator>::value_type::first_type>,
                             typename iterator_traits<InputIterator>::value_type::second_type>    // exposition only
```

At the end of the definition of class `map` in §23.4.4.1 [map.overview], add the following *deduction-guide*s:

```
  template <class K>
    pair<const_iterator, const_iterator> equal_range(const K& x) const;
};

template <class InputIterator,
      class Compare = less<iter_key_t<InputIterator>>,
      class Allocator = allocator<iter_to_alloc_t<InputIterator>>>
map(InputIterator, InputIterator, Compare = Compare(), Allocator = Allocator())
  -> map<iter_key_t<InputIterator>, iter_val_t<InputIterator>, Compare, Allocator>;

template<class Key, class T, class Compare = less<Key>, class Allocator = allocator<pair<const Key, T>>>
map(initializer_list<pair<const Key, T>>, Compare = Compare(), Allocator = Allocator())
  -> map<Key, T, Compare, Allocator>;

template <class InputIterator, class Allocator>
map(InputIterator, InputIterator, Allocator)
  -> map<iter_key_t<InputIterator>, iter_val_t<InputIterator>, less<iter_key_t<InputIterator>>, Allocator>;

template<class Key, class T, class Allocator>
map(initializer_list<pair<const Key, T>>, Allocator) -> map<Key, T, less<Key>, Allocator>;
```

At the end of the definition of class `multimap` in §23.4.5.1 [multimap.overview], add the following *deduction-guide*s:

```
  template <class K>
    pair<const_iterator, const_iterator> equal_range(const K& x) const;
};

template <class InputIterator,
      class Compare = less<iter_key_t<InputIterator>>,
      class Allocator = allocator<iter_to_alloc_t<InputIterator>>>
multimap(InputIterator, InputIterator, Compare = Compare(), Allocator = Allocator())
  -> multimap<iter_key_t<InputIterator>, iter_val_t<InputIterator>, Compare, Allocator>;

template<class Key, class T, class Compare = less<Key>, class Allocator = allocator<pair<const Key, T>>>
multimap(initializer_list<pair<const Key, T>>, Compare = Compare(), Allocator = Allocator())
  -> multimap<Key, T, Compare, Allocator>;

template <class InputIterator, class Allocator>
multimap(InputIterator, InputIterator, Allocator)
```

```
  -> multimap<iter_key_t<InputIterator>, iter_val_t<InputIterator>, less<iter_key_t<InputIterator>>, Allocator>;

template<class Key, class T, class Allocator>
multimap(initializer_list<pair<const Key, T>>, Allocator) -> multimap<Key, T, less<Key>, Allocator>;
```

At the end of the definition of class `set` in §23.4.6.1 [set.overview], add the following *deduction-guides*:

```
  template <class K>
    pair<const_iterator, const_iterator> equal_range(const K& x) const;
};

template <class InputIterator,
      class Compare = less<typename iterator_traits<InputIterator>::value_type>,
      class Allocator = allocator<typename iterator_traits<InputIterator>::value_type>>
set(InputIterator, InputIterator,
    Compare = Compare(), Allocator = Allocator())
  -> set<typename iterator_traits<InputIterator>::value_type, Compare, Allocator>;

template<class Key, class Compare = less<Key>, class Allocator = allocator<Key>>
set(initializer_list<Key>, Compare = Compare(), Allocator = Allocator())
  -> set<Key, Compare, Allocator>;

template<class InputIterator, class Allocator>
set(InputIterator, InputIterator, Allocator)
  -> set<typename iterator_traits<InputIterator>::value_type,
         less<typename iterator_traits<InputIterator>::value_type>, Allocator>;

template<class Key, class Allocator>
set(initializer_list<Key>, Allocator) -> set<Key, less<Key>, Allocator>;
```

At the end of the definition of class `multiset` in §23.4.7.1 [multiset.overview], add the following *deduction-guide*:

```
  template <class K>
    pair<const_iterator, const_iterator> equal_range(const K& x) const;
};

template <class InputIterator,
      class Compare = less<typename iterator_traits<InputIterator>::value_type>,
      class Allocator = allocator<typename iterator_traits<InputIterator>::value_type>>
multiset(InputIterator, InputIterator,
    Compare = Compare(), Allocator = Allocator())
  -> multiset<typename iterator_traits<InputIterator>::value_type, Compare, Allocator>;

template<class Key, class Compare = less<Key>, class Allocator = allocator<Key>>
multiset(initializer_list<Key>, Compare = Compare(), Allocator = Allocator())
  -> multiset<Key, Compare, Allocator>;

template<class InputIterator, class Allocator>
multiset(InputIterator, InputIterator, Allocator)
  -> multiset<typename iterator_traits<InputIterator>::value_type,
         less<typename iterator_traits<InputIterator>::value_type>, Allocator>;

template<class Key, class Allocator>
multiset(initializer_list<Key>, Allocator) -> multiset<Key, less<Key>, Allocator>;
```

§23.5 [unord]

Add a paragraph to the end of §23.5.1 [unord.general]:

> The exposition only type aliases `iter_key_t`, `iter_val_t`, and `iter_to_alloc_t` defined in [associative.general] may appear in deduction guides for unordered containers

At the end of the definition of class `unordered_map` in §23.5.4.1 [unord.map.overview], add the following *deduction-guides*:

```
  void reserve(size_type n);
};

template<class InputIterator,
         class Hash = hash<iter_key_t<InputIterator>>, class Pred = equal_to<iter_key_t<InputIterator>>,
         class Allocator = allocator<iter_to_alloc_t<InputIterator>>>
unordered_map(InputIterator, InputIterator, typename see below::size_type = see below,
              Hash = Hash(), Pred = Pred(), Allocator = Allocator())
  -> unordered_map<iter_key_t<InputIterator>, iter_value_t<InputIterator>, Hash, Pred, Allocator>;

template<class Key, class T, class Hash = hash<Key>,
         class Pred = equal_to<Key>, class Allocator = allocator<pair<const Key, T>>>
unordered_map(initializer_list<pair<const Key, T>>, typename see below::size_type = see below,
```

```
                Hash = Hash(), Pred = Pred(), Allocator = Allocator())
  -> unordered_map<Key, T, Hash, Pred, Allocator>;

template<class InputIterator, class Allocator>
unordered_map(InputIterator, InputIterator, typename see below::size_type, Allocator)
    -> unordered_map<iter_key_t<InputIterator>, iter_val_t<InputIterator>,
          hash<iter_key_t<InputIterator>>, equal_to<iter_key_t<InputIterator>>, Allocator>;

template<class InputIterator, class Allocator>
unordered_map(InputIterator, InputIterator, Allocator)
    -> unordered_map<iter_key_t<InputIterator>, iter_val_t<InputIterator>,
          hash<iter_key_t<InputIterator>>, equal_to<iter_key_t<InputIterator>>, Allocator>;

template<class InputIterator, class Hash, class Allocator>
unordered_map(InputIterator, InputIterator, typename see below::size_type, Hash, Allocator)
    -> unordered_map<iter_key_t<InputIterator>, iter_val_t<InputIterator>, Hash,
            equal_to<iter_key_t<InputIterator>>, Allocator>;

template<class Key, class T, typename Allocator>
unordered_map(initializer_list<pair<const Key, T>>, typename see below::size_type, Allocator)
     -> unordered_map<Key, T, hash<Key>, equal_to<Key>, Allocator>;

template<class Key, class T, typename Allocator>
unordered_map(initializer_list<pair<const Key, T>>, Allocator)
     -> unordered_map<Key, T, hash<Key>, equal_to<Key>, Allocator>;

template<class Key, class T, class Hash, class Allocator>
unordered_map(initializer_list<pair<const Key, T>>, typename see below::size_type, Hash, Allocator)
  -> unordered_map<Key, T, Hash, equal_to<Key>, Allocator>;
```

Add the following paragraph to the end of §23.5.4.1 [unord.map.overview]:

> A `size_type` parameter type in an `unordered_map` deduction guide refers to the `size_type` member type of the type deduced by the deduction guide.

At the end of the definition of class `unordered_multimap` in §23.5.5.1 [unord.multimap.overview], add the following *deduction-guide*s

```
  void reserve(size_type n);
};

template<class InputIterator,
         class Hash = hash<iter_key_t<InputIterator>>, class Pred = equal_to<iter_key_t<InputIterator>>,
         class Allocator = allocator<iter_to_alloc_t<InputIterator>>>
unordered_multimap(InputIterator, InputIterator, typename see below::size_type = see below,
                Hash = Hash(), Pred = Pred(), Allocator = Allocator())
    -> unordered_multimap<iter_key_t<InputIterator>, iter_value_t<InputIterator>, Hash, Pred, Allocator>;

template<class Key, class T, class Hash = hash<Key>,
          class Pred = equal_to<Key>, class Allocator = allocator<pair<const Key, T>>>
unordered_multimap(initializer_list<pair<const Key, T>>, typename see below::size_type = see below,
                Hash = Hash(), Pred = Pred(), Allocator = Allocator())
  -> unordered_multimap<Key, T, Hash, Pred, Allocator>;

template<class InputIterator, class Allocator>
unordered_multimap(InputIterator, InputIterator, typename see below::size_type, Allocator)
    -> unordered_multimap<iter_key_t<InputIterator>, iter_val_t<InputIterator>,
          hash<iter_key_t<InputIterator>>, equal_to<iter_key_t<InputIterator>>, Allocator>;

template<class InputIterator, class Allocator>
unordered_multimap(InputIterator, InputIterator, Allocator)
    -> unordered_multimap<iter_key_t<InputIterator>, iter_val_t<InputIterator>,
          hash<iter_key_t<InputIterator>>, equal_to<iter_key_t<InputIterator>>, Allocator>;

template<class InputIterator, class Hash, class Allocator>
unordered_multimap(InputIterator, InputIterator, typename see below::size_type, Hash, Allocator)
    -> unordered_multimap<iter_key_t<InputIterator>, iter_val_t<InputIterator>, Hash,
            equal_to<iter_key_t<InputIterator>>, Allocator>;

template<class Key, class T, typename Allocator>
unordered_multimap(initializer_list<pair<const Key, T>>, typename see below::size_type, Allocator)
     -> unordered_multimap<Key, T, hash<Key>, equal_to<Key>, Allocator>;

template<class Key, class T, typename Allocator>
unordered_multimap(initializer_list<pair<const Key, T>>, Allocator)
     -> unordered_multimap<Key, T, hash<Key>, equal_to<Key>, Allocator>;
```

```
  template<class Key, class T, class Hash, class Allocator>
  unordered_multimap(initializer_list<pair<const Key, T>>, typename see below::size_type, Hash, Allocator)
    -> unordered_multimap<Key, T, Hash, equal_to<Key>, Allocator>;
```

Add the following paragraph to the end of §23.5.5.1 [unord.multimap.overview]:

A `size_type` parameter type in an `unordered_multimap` deduction guide refers to the `size_type` member type of the type deduced by deduction guide.

At the end of the definition of class `unordered_set` in §23.5.6.1 [unord.set.overview], add the following *deduction-guide*s:

```
    void reserve(size_type n);
  };

  template<class InputIterator,
           class Hash = hash<typename iterator_traits<InputIterator>::value_type>,
           class Pred = equal_to<typename iterator_traits<InputIterator>::value_type>,
           class Allocator = allocator<typename iterator_traits<InputIterator>::value_type>>
  unordered_set(InputIterator, InputIterator, typename see below::size_type = see below,
                Hash = Hash(), Pred = Pred(), Allocator = Allocator())
    -> unordered_set<typename iterator_traits<InputIterator>::value_type,
                     Hash, Pred, Allocator>;

  template<class T, class Hash = hash<T>,
           class Pred = equal_to<T>, class Allocator = allocator<T>>
  unordered_set(initializer_list<T>, typename see below::size_type = see below,
                Hash = Hash(), Pred = Pred(), Allocator = Allocator())
    -> unordered_set<T, Hash, Pred, Allocator>;

  template<class InputIterator,  class Allocator>
  unordered_set(InputIterator, InputIterator, typename see below::size_type, Allocator)
    -> unordered_set<typename iterator_traits<InputIterator>::value_type,
                     hash<typename iterator_traits<InputIterator>::value_type>,
                     equal_to<typename iterator_traits<InputIterator>::value_type>,
                     Allocator>;

  template<class InputIterator, class Hash, class Allocator>
  unordered_set(InputIterator, InputIterator, typename see below::size_type,
                Hash, Allocator)
    -> unordered_set<typename iterator_traits<InputIterator>::value_type, Hash,
                     equal_to<typename iterator_traits<InputIterator>::value_type>,
                     Allocator>;

  template<class T, class Allocator>
  unordered_set(initializer_list<T>, typename see below::size_type, Allocator)
    -> unordered_set<T, hash<T>, equal_to<T>, Allocator>;

  template<class T, class Hash, class Allocator>
  unordered_set(initializer_list<T>, typename see below::size_type, Hash, Allocator)
    -> unordered_set<T, Hash, equal_to<T>, Allocator>;
```

Add the following paragraph to the end of §23.5.6.1 [unord.set.overview]:

A `size_type` parameter type in an `unordered_set` deduction guide refers to the `size_type` member type of the primary `unordered_set` template.

At the end of the definition of class `unordered_multiset` in §23.5.7.1 [unord.multiset.overview], add the following *deduction-guide*s:

```
    void reserve(size_type n);
  };

  template<class InputIterator,
           class Hash = hash<typename iterator_traits<InputIterator>::value_type>,
           class Pred = equal_to<typename iterator_traits<InputIterator>::value_type>,
           class Allocator = allocator<typename iterator_traits<InputIterator>::value_type>>
  unordered_multiset(InputIterator, InputIterator, see below::size_type = see below,
                Hash = Hash(), Pred = Pred(), Allocator = Allocator())
    -> unordered_multiset<typename iterator_traits<InputIterator>::value_type,
                     Hash, Pred, Allocator>;

  template<class T, class Hash = hash<T>,
           class Pred = equal_to<T>, class Allocator = allocator<T>>
  unordered_multiset(initializer_list<T>, typename see below::size_type = see below,
                Hash = Hash(), Pred = Pred(), Allocator = Allocator())
    -> unordered_multiset<T, Hash, Pred, Allocator>;
```

```
template<class InputIterator,  class Allocator>
unordered_multiset(InputIterator, InputIterator, typename see below::size_type, Allocator)
  -> unordered_multiset<typename iterator_traits<InputIterator>::value_type,
                  hash<typename iterator_traits<InputIterator>::value_type>,
                  equal_to<typename iterator_traits<InputIterator>::value_type>,
                  Allocator>;

template<class InputIterator,  class Hash, class Allocator>
unordered_multiset(InputIterator, InputIterator, typename see below::size_type,
               Hash, Allocator)
  -> unordered_multiset<typename iterator_traits<InputIterator>::value_type, Hash,
                  equal_to<typename iterator_traits<InputIterator>::value_type>, Allocator>;

template<class T, class Allocator>
unordered_multiset(initializer_list<T>, typename see below::size_type, Allocator)
  -> unordered_multiset<T, hash<T>, equal_to<T>, Allocator>;

template<class T, class Hash, class Allocator>
unordered_multiset(initializer_list<T>, typename see below::size_type, Hash, Allocator)
  -> unordered_multiset<T, Hash, equal_to<T>, Allocator>;
```

Add the following paragraph to the end of §23.5.7.1 [unord.multiset.overview]:

> A `size_type` parameter type in an `unordered_multiset` deduction guide refers to the `size_type` member type of the primary `unordered_multiset` template.

§23.6 [container.adaptors]

At the end of §23.6.1 [container.adaptors.general], and the following paragraph

> A deduction guide for a container adaptor shall not participate in overload resolution if any of the following are true:
>
> - It has an `InputIterator` template parameter that is called with a type that does not qualify as an input iterator
> - It has a `Compare` template parameter that is called with a type that qualifies as an allocator
> - It has a `Container` template parameter that is called with a type that qualifies as an allocator
> - It has an `Allocator` template parameter that is called with a type that does not qualify as an allocator
> - It has both `Container` and `Allocator` template parameters, and `uses_allocator_v<Container, Allocator>` is `false`

At the end of the definition of class `queue` in §23.6.4.1 [queue.defn] insert

```
  void swap(queue& q) noexcept(is_nothrow_swappable_v<Container>)
    { using std::swap; swap(c, q.c); }
};

template<class Container>
queue(Container) -> queue<typename Container::value_type, Container>;

template<class Container, class Allocator>
queue(Container, Allocator) -> queue<typename Container::value_type, Container>;
```

At the end of the definition of class `priority_queue` in §23.6.5 [priority.queue], add the following *deduction-guide*s:

```
  void swap(priority_queue& q) noexcept(is_nothrow_swappable_v<Container> &&
                                        is_nothrow_swappable_v<Compare>)
    { using std::swap; swap(c, q.c); swap(comp, q.comp); }
  };

template <class Compare, class Container>
priority_queue(Compare, Container)
  -> priority_queue<typename Container::value_type, Container, Compare>;

template<class InputIterator,
         class Compare = less<typename iterator_traits<InputIterator>::value_type>,
         class Container = vector<typename iterator_traits<InputIterator>::value_type>>
priority_queue(InputIterator, InputIterator, Compare = Compare(), Container = Container())
        -> priority_queue<typename iterator_traits<InputIterator>::value_type, Container, Compare>;

template<class Compare, class Container, class Allocator>
priority_queue(Compare, Container, Allocator)
  -> priority_queue<typename Container::value_type, Container, Compare>;
```

At the end of the definition of class `stack` in §23.6.6.1 [stack.defn], add

```
  void swap(stack& q) noexcept(is_nothrow_swappable_v<Container>)
    { using std::swap; swap(c, q.c); }
};

template<class Container>
stack(Container) -> stack<typename Container::value_type, Container>;

template<class Container, class Allocator>
stack(Container, Allocator) -> stack<typename Container::value_type, Container>;
```

§24 [iterators]

No changes are required in clause 24 as the implicitly generated deduction guides provide the necessary deduction.

§25 [algorithms]

No changes are required in clause 25 due to lack of template classes in this clause.

§26 [numerics]

§26.5 [complex.numbers] Class `complex` does not require exicit deduction guides as the implicity generated deduction guides provide the necessary deduction.

§26.6 [rand]

This section does not require explicit deduction guides. The random number engines (§ 26.6.3 [rand.eng]) all have non-type template parameters and are therefore not suitable for template parameter deduction. All of the distributions (§26.6.8 [rand.dist]) that could in principle be deduced from constructor arguments are properly deduced by their implicitly generated deduction guides.

§26.7 [numarray]

We consider `valarray`. We first note that the implicit deduction guides imply the following:

```
int iar[] = {1, 2, 3};
int *ip = iarr;
        valarray va(ip, 3); // Deduces valarray<int>
```

The point is that the `valarray<T>::valarray(const T *, size_t)` constructor is a better match than the `valarray<T>::valarray(const T &, size_t)` constructor. We think this is preferable because we believe that is the much more common default. Note that P0091R4 discusses a language extension (`= delete` for the `valarray(const T *, size_t)` deduction guide) that would facilitate suppressing deduction in this case. However, as above, we believe the appropriate deduction is produced by the implicitly-generated deduction guides.

We do add a deduction guide to enable the following deduction:

```
int iarr[] = {1, 2, 3};
valarray va(iarr, 3); // Needs explicit deduction guide to deduce valarray<int>
```

At the end of the definition of class `valarray` in §26.7.2.1 [template.valarray.overview], insert the following

```
  void resize(size_t sz, T c = T());
};

template<typename T, size_t cnt> valarray(const T(&)[cnt], size_t) -> valarray<T>;
```

§27 [input.output]

No changes are required in clause 27 as the implicitly generated deduction guides provide the necessary deduction.

§28 [re]

At the end of the definition of class `basic_regex` in §28.8 [re.regex], insert the following:

```
  // 28.8.6, swap
  void swap(basic_regex&);
};
```

```
  template<class ForwardIterator>
  basic_regex(ForwardIterator, ForwardIterator, regex_constants::syntax_option_type = regex_constants::ECMAScript)
    -> basic_regex<typename iterator_traits<ForwardIterator>::value_type>;
```

§29 [atomics]

No changes are required in clause 29 as the implicitly generated deduction guides provide the necessary deduction.

§30 [thread]

At the end of the definition class `lock_guard` in [thread.lock.guard], add

```
  };

template<class M> lock_guard(lock_guard<M>) -> lock_guard<M>;
```

At the end of the definition class `scoped_lock` in [thread.lock.scoped], add

```
  };

template<class... M> scoped_lock(scoped_lock<M...>) -> scoped_lock<M...>;
```

At the end of the definition class `unique_lock` in [thread.lock.unique], add

```
  };

template<class M> unique_lock(unique_lock<M>) -> unique_lock<M>;
```

At the end of the definition class `shared_lock` in [thread.lock.shared], add

```
  };

template<class M> shared_lock(shared_lock<M>) -> shared_lock<M>;
```

Feature test macro

The recommended feature test macro for this feature is `__cpp_lib_deduction_guides`